

Decision Making & Looping Statements

Module 3 - Control the flow of a Java program

Decide

Repeat

Control

A PROGRAM CAN CHOOSE

```
if (mark >= 50) {  
    println("Pass");  
} else {  
    println("Try Again");  
}
```

A PROGRAM CAN REPEAT

```
for (int i = 1; i <= 3; i++) {  
    println(i);  
}
```

Learn - Visualize - Code - Practice

What you will learn

Control flow determines **which code runs**, **how many times it runs**, and **when execution stops or skips ahead**.

DECISION MAKING

Choose a path

Use **if**, **if-else**, **else-if**, nested **if** and **switch**.

LOOPING

Repeat a task

Use **for**, **while**, **do-while** and enhanced **for**.

CONTROL TRANSFER

Change normal execution

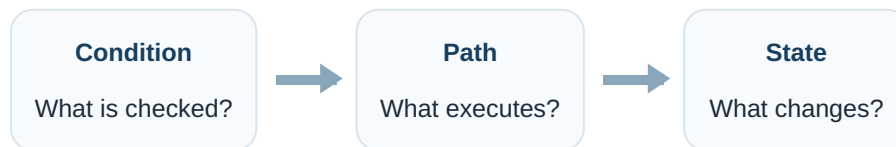
break stops a loop or switch, **continue** skips an iteration, and **return** exits a method.

REASONING

Trace before running

Predict which branch executes and how loop variables change on every iteration.

Recommended study pattern

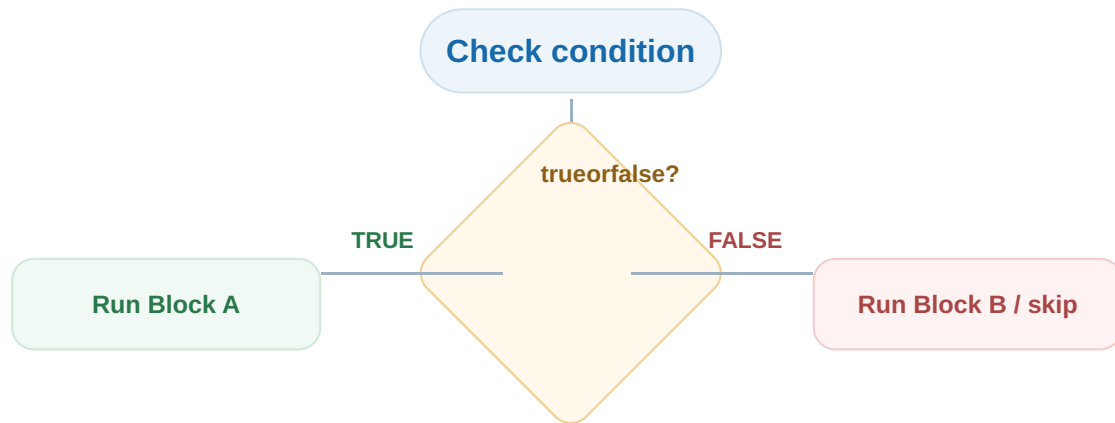


ACCURACY NOTE

In this guide, **return** is taught as a **control-transfer statement**, not as a looping statement. It ends the current method immediately.

What is decision making?

Decision-making statements allow a program to evaluate a **condition** and execute different code depending on whether that condition is **true** or **false**.



CONDITION

age >= 18

Java evaluates this expression and produces a boolean result: true or false.

REAL-WORLD CONNECTION

Login system

If credentials are valid, open the dashboard; otherwise, show an error.

Java decision tools

if

if-else

else-if ladder

nested if

switch

Quick Check: If `int age = 16`, what is the result of `age >= 18`?

The if statement

Use **if** when a block should run **only when a condition is true**. If the condition is false, that block is skipped.

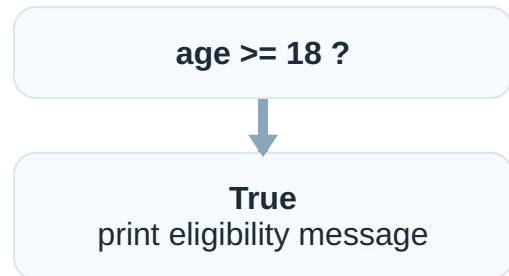
SYNTAX

```
if (condition) {  
    // statements  
}
```

THINK OF IT AS

IF this is true, **THEN** do this.

EXECUTION FLOW



Example

```
public class Example {  
    public static void main(String[] args) {  
        int age = 18;  
  
        if (age >= 18) {  
            System.out.println("Eligible to vote");  
        }  
    }  
}
```

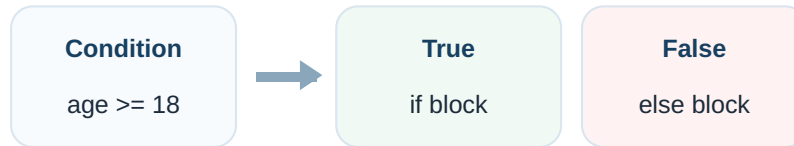
Eligible to vote

COMMON MISTAKE

Do not place a semicolon immediately after the condition: **if (age >= 18);** changes the logic and is a common beginner bug.

The if-else statement

Use **if-else** when there are **two possible outcomes**. Exactly one branch is selected.



```
public class Example {  
    public static void main(String[] args) {  
        int age = 16;  
  
        if (age >= 18) {  
            System.out.println("Eligible to vote");  
        } else {  
            System.out.println("Not eligible to vote");  
        }  
    }  
}
```

Not eligible to vote

WHEN TRUE

The **if** block runs. The else block is skipped.

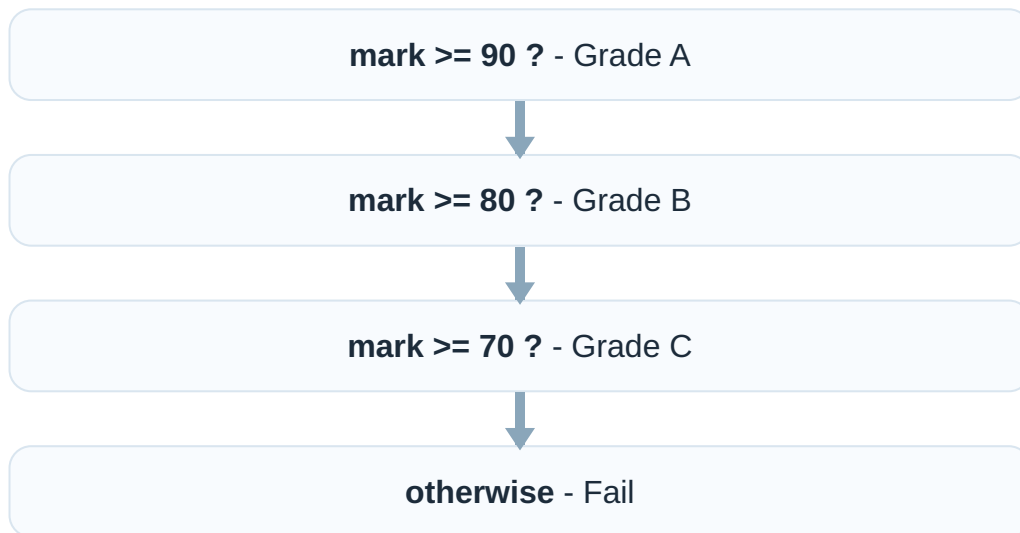
WHEN FALSE

The **else** block runs. The if block is skipped.

Predict: Change **age** to 25. Which message appears?

The else-if ladder

Use an **else-if ladder** when there are multiple conditions. Java checks from top to bottom and executes the **first true branch**; the remaining branches are skipped.



```
int mark = 85;

if (mark >= 90) {
    System.out.println("Grade A");
} else if (mark >= 80) {
    System.out.println("Grade B");
} else if (mark >= 70) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

Grade B

WHY ORDER MATTERS

Put the **most specific / highest threshold first**. If you check `mark >= 70` before `mark >= 90`, a mark of 95 would match the earlier condition too soon.

Nested if

A **nested if** is an if statement placed inside another if statement. It is useful when a second check should happen **only after** the first check succeeds.

ATM EXAMPLE

```
int pin = 1234;
int balance = 1000;
int withdraw = 500;

if (pin == 1234) {
    if (balance >= withdraw) {
        System.out.println("Successful");
    }
}
```

HOW IT WORKS

1. Check whether the PIN is correct.
2. Only then check whether enough balance exists.
3. If both checks pass, complete the action.

Visual relationship

PIN correct?



Balance enough?



Withdraw

READABILITY WARNING

Deeply nested conditions become difficult to read. At beginner level, first understand the logic; later you will learn ways to simplify complex nesting.

The switch statement

Use **switch** when one expression can match one of several **fixed choices**. It can be clearer than a long chain of equality checks.

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

Wednesday

CASE

Represents one possible matching value.

BREAK

Stops the switch so execution does not fall through to the next case.

DEFAULT

Runs when no case matches.

BEST FIT

Menus, day numbers, fixed commands, status codes.

Modern switch expression

Modern Java supports a concise **switch expression** using `->`. It can directly produce a value and avoids traditional fall-through behavior.

```
int day = 3;

String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    case 4 -> "Thursday";
    case 5 -> "Friday";
    case 6 -> "Saturday";
    case 7 -> "Sunday";
    default -> "Invalid";
};

System.out.println(name);
```

Wednesday

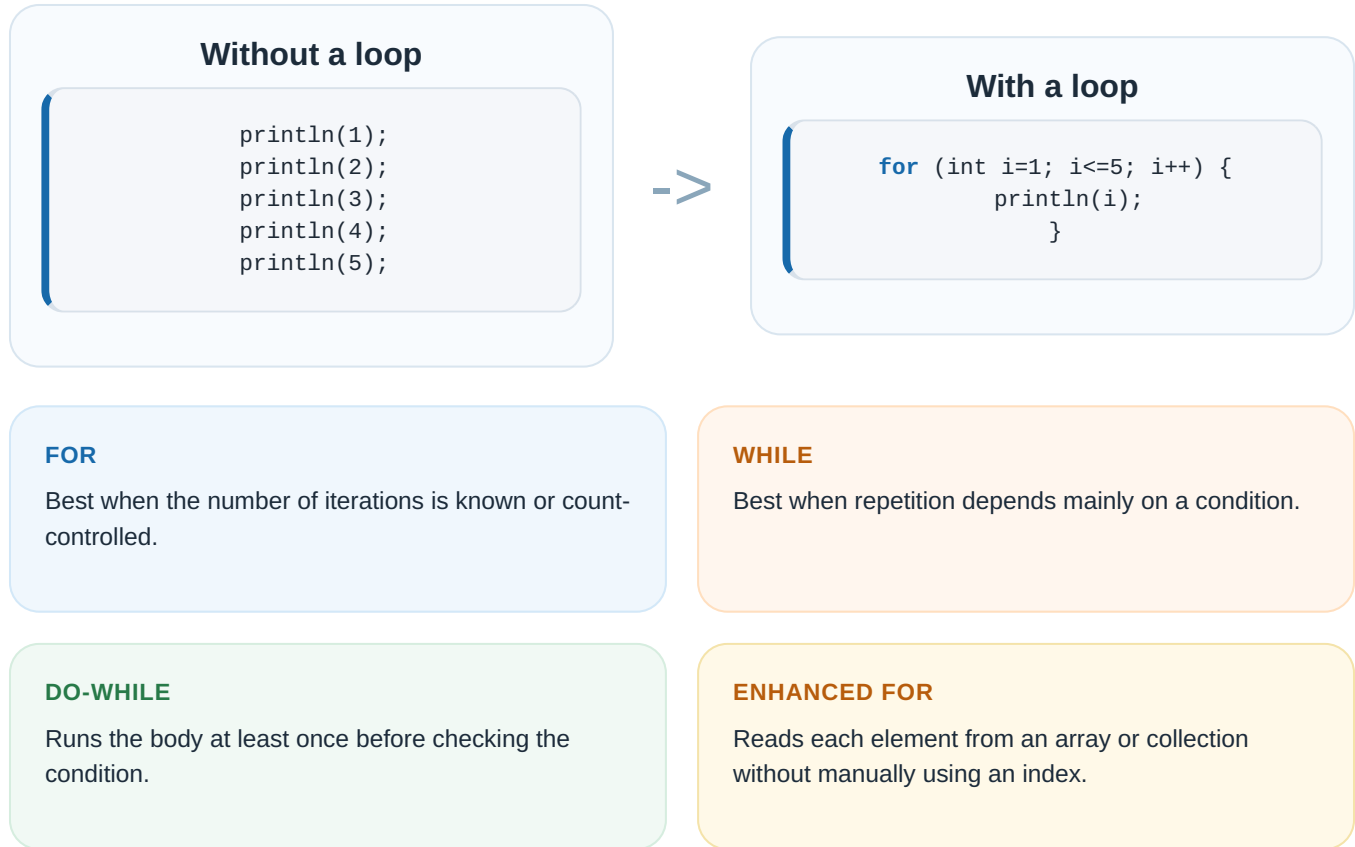
Traditional switch	Modern switch expression
Uses case value :	Uses case value ->
Often needs break	No normal fall-through between arrow cases
Usually executes statements	Can directly produce a value

VERSION NOTE

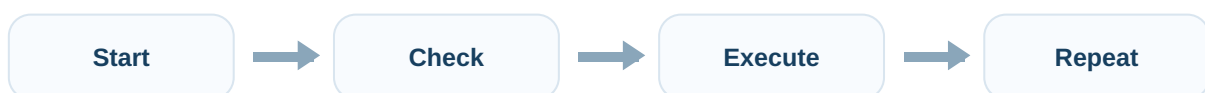
Switch expressions became a standard Java feature in **Java 14**. If your lab uses an older JDK, the traditional form may be required.

Why do we need loops?

Loops execute a block of code repeatedly while a condition allows the repetition to continue.



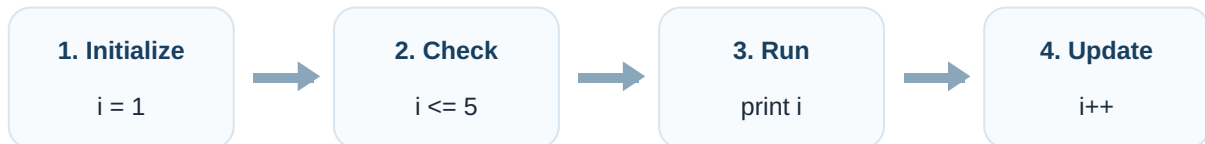
The loop idea



The for loop

Use a **for loop** when the number of iterations is known or naturally controlled by a counter.

```
for (initialization; condition; update) {
    // statements
}
```



```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

```
1
2
3
4
5
```

TRACE ONE ITERATION

At **i = 3**: check **3 <= 5** - true - print 3 - execute **i++** - i becomes 4.

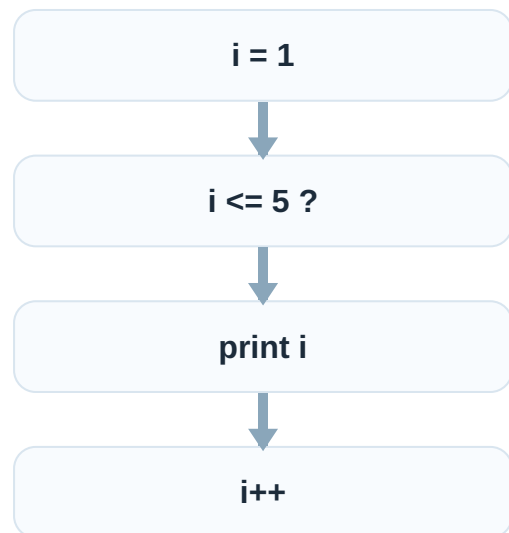
The while loop

A **while loop** keeps running as long as its condition remains true. The condition is checked **before** each iteration.

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

```
1
2
3
4
5
```



COMMON MISTAKE - INFINITE LOOP

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    // forgot i++
}
```

Because **i** never changes, the condition never becomes false.

The do-while loop

A **do-while** loop executes its body **at least once** because the condition is checked only after the body runs.

```
do {
    // statements
} while (condition);
```

```
int i = 6;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

6

WHY 6 IS PRINTED

Step 1: body runs immediately.

Step 2: 6 is printed.

Step 3: i becomes 7.

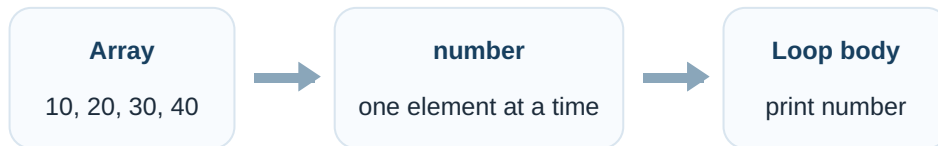
Step 4: check 7 <= 5 - false - stop.

while	do-while
Checks condition first	Executes body first
May execute zero times	Executes at least once

Quick Check: Which loop is useful when a menu should be shown before asking whether the user wants to continue?

Enhanced for loop (for-each)

The enhanced **for** loop reads each element from an array or collection without manually managing an index.



```

int[] numbers = {10, 20, 30, 40};

for (int number : numbers) {
    System.out.println(number);
}
  
```

```

10
20
30
40
  
```

ADVANTAGE

Simple and readable when you only need each value.

LIMITATION

You do not directly get the element's index from the loop syntax.

READ THE SYNTAX ALOUD

For each **int number** inside **numbers**, execute the block.

break and continue

Both statements change normal loop execution, but they do very different jobs.

BREAK - STOP COMPLETELY

```
for (int i=1; i<=10; i++) {  
    if (i == 6) {  
        break;  
    }  
    System.out.println(i);  
}
```

1 2 3 4 5

CONTINUE - SKIP ONE ITERATION

```
for (int i=1; i<=5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

1 2 4 5

Statement	Effect	Where used
break	Terminates the nearest loop or switch	Loops and switch
continue	Skips the rest of the current iteration	Loops

Choose: You are searching an array and already found the required value. Would **break** or **continue** usually make more sense?

The return statement

return ends execution of the current method. It may also send a value back to the caller.

RETURN A VALUE

```
static int add(int a, int b) {  
    return a + b;  
}  
  
int result = add(10, 20);  
System.out.println(result);
```

30

EXIT EARLY

```
static void checkAge(int age) {  
    if (age < 18) {  
        System.out.println("Not  
Eligible");  
        return;  
    }  
    System.out.println("Eligible");  
}
```

checkAge(16) -> Not Eligible

KEY IDEA

After **return** executes, the method stops immediately. In the second example, "**Eligible**" is never printed for age 16.

DO NOT CONFUSE

break exits a loop/switch. **return** exits the **entire current method**.

Which control statement should I use?

Need	Best starting choice	Reason
Run code only when something is true	if	Single conditional branch
Choose between two outcomes	if-else	True path + false path
Check several ranges or conditions	else-if ladder	Top-to-bottom conditional testing
Choose one of fixed matching values	switch	Clear value-based choices
Repeat a known number of times	for	Counter fits naturally
Repeat while a condition stays true	while	Condition-driven repetition
Run at least once, then decide	do-while	Condition checked after body
Visit every array/collection element	enhanced for	No manual index required
Stop a loop/switch now	break	Immediate termination
Skip only this loop iteration	continue	Moves to next iteration
Stop a method / send back a value	return	Method-level control transfer

MEMORY SHORTCUT

Decision = choose. Loop = repeat. break = stop loop/switch. continue = skip once. return = leave method.

Check your understanding

1. Predict: What prints when **age = 15** in an if-else eligibility program?

2. Reason: Why should **mark >= 90** appear before **mark >= 70** in a grade ladder?

3. Predict: How many times does **for (int i=0; i<3; i++)** execute?

4. Compare: What is the one guaranteed execution difference between while and do-while?

5. Choose: Which statement skips only the current loop iteration?

MINI CHALLENGE

Write a program that reads a mark and prints:

A for 90+, **B** for 80-89, **C** for 70-79, otherwise **Needs Improvement**.

LOOP CHALLENGE

Print numbers from 1 to 10, but skip 5 and stop completely when the number reaches 8.

INTERVIEW CONNECTION

Beginner interviews often ask the difference between **while vs do-while**, **break vs continue**, and when **switch** is preferable to a long conditional chain.

Quick recap

if - one condition

if-else - two paths

else-if - many conditions

switch - fixed choices

for - counted repetition

while - check first

do-while - run first

break - stop

continue - skip

return - leave method

NEXT LEARNING HABIT

For every control-flow program, trace values on paper before running it. That habit builds debugging skill much faster than memorizing syntax.